Automatic Conflict Resolution System for Notes

📌 Abstract

This project aims to design and implement an automatic conflict resolution system for a note-taking application. Conflicts occur when multiple edits are made to the same note across different devices without proper synchronization. The proposed system will automatically merge changes wherever possible while preserving all user data. In cases where automatic merging is not feasible, the system will provide assisted conflict resolution with clear summaries and suggestions.

📖 Introduction

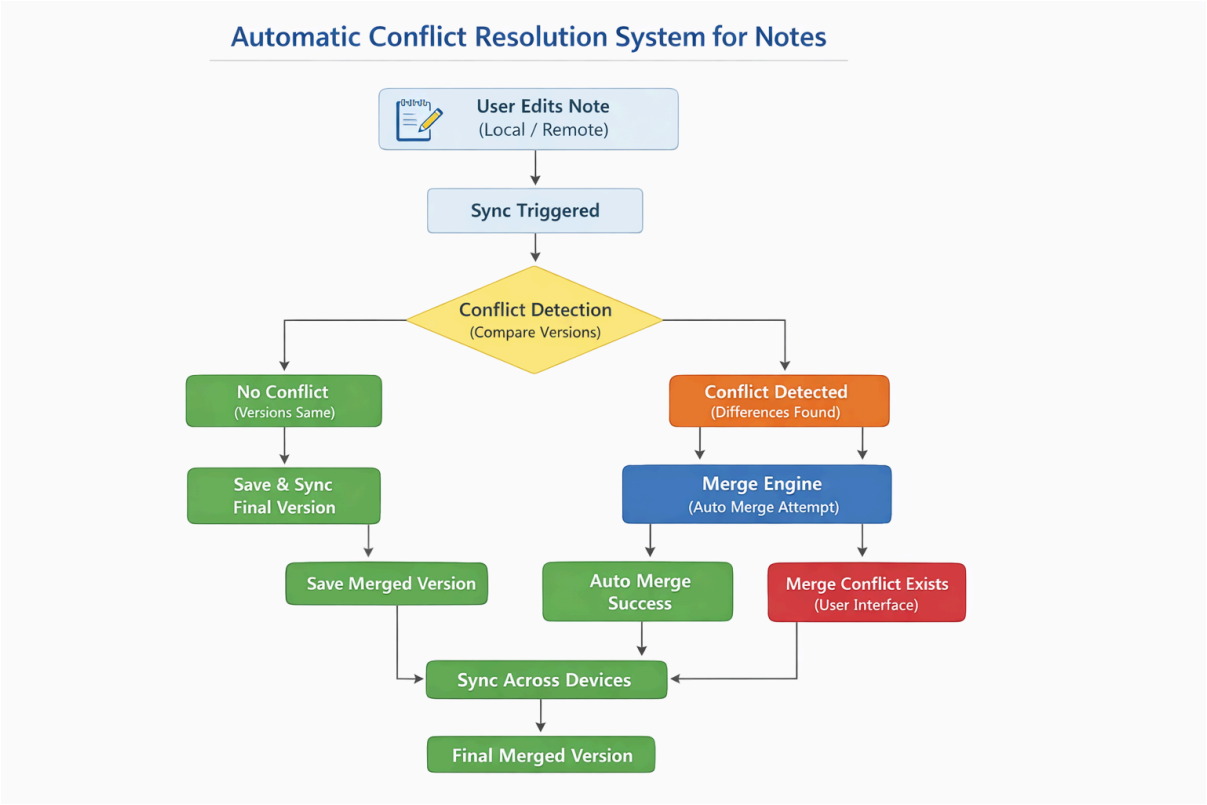
Modern note-taking applications rely heavily on synchronization between local and remote storage. However, when users edit notes offline or across multiple devices, conflicts may arise due to version mismatches.

Currently, resolving these conflicts requires manual intervention, which can be time-consuming and error-prone, especially for large notes. This project proposes an intelligent system to automate conflict detection and resolution, improving user experience and ensuring data consistency.

🎯 Objectives

- Detect conflicts between local and remote note versions

- Automatically merge non-overlapping changes
- Provide assisted conflict resolution for complex cases
- Ensure no data loss during merging
- Maintain seamless synchronization across devices



⚙️ System Design

1. Architecture Overview

The system consists of three main components:

- **Conflict Detection Module**
- **Merge Engine**
- **User Interface for Assisted Resolution**

2. Workflow

1. User edits note locally

2. Sync process detects version mismatch
3. Conflict detection module identifies differences
4. Merge engine attempts automatic resolution
5. If unresolved, UI presents assisted resolution options

🧠 Algorithm Design

1. Conflict Detection

- Compare local and remote versions of notes
- Use line-by-line or block-based diff algorithm
- Identify:
  - Insertions
  - Deletions
  - Modifications

2. Automatic Merge Strategy

- If changes occur in different sections → merge automatically
- If overlapping changes exist:
  - Preserve both versions
  - Mark conflict region

3. Assisted Conflict Resolution

- Display:
  - Original version
  - Local changes
  - Remote changes
- Provide:
  - Suggested merge
  - Highlighted conflicts
  - Warning for potential data loss

4. Optional AI Enhancement (Future Scope)

- Use NLP techniques to intelligently merge text
- Suggest context-aware resolutions

🛠️ Tech Stack

- **Language:** TypeScript
- **Frontend:** HTML, CSS, JavaScript

- **Version Control Concepts:** Git-based workflows
- **Algorithms:** Diff and merge algorithms

💻 Sample Implementation (Basic Example)

```
function mergeNotes(local: string, remote: string): string {
  if (local === remote) {
    return local;
  }

  // Simple merge logic
  let merged = "";

  const localLines = local.split("\n");
  const remoteLines = remote.split("\n");

  const maxLength = Math.max(localLines.length, remoteLines.length);

  for (let i = 0; i < maxLength; i++) {
    if (localLines[i] === remoteLines[i]) {
      merged += (localLines[i] || "") + "\n";
    } else {
      merged += `<<<<<<< LOCAL\n${localLines[i] || ""}\n=======\n${remoteLines[i] ||
""}\n>>>>>>> REMOTE\n`;
    }
  }

  return merged;
}
```

🚀 Future Enhancements

- AI-based smart merging
- Real-time collaboration support
- Advanced visualization of conflicts

📌 Conclusion

This project will significantly improve the usability of note-taking applications by automating conflict resolution and minimizing user effort. By combining efficient algorithms with a user-friendly interface, it ensures reliable synchronization and enhanced user experience across platforms.